3 Machine Learning Workflows

Machine learning workflows describe the sequence of steps used to transform raw engineering data into a trained and evaluated predictive model. Although the learning algorithm is important, successful machine learning applications often depend just as much on the quality of the data, the choice of features, and the way the model is evaluated. A well-organized workflow helps ensure that the model learns meaningful patterns rather than noise or accidental structure in the data.

Figure 3.1 illustrates a common machine learning workflow. The process begins by collecting data from experiments, sensors, simulations, or existing records. The data are then inspected and cleaned to identify missing values, outliers, inconsistent units, or other quality issues. After this preparation step, feature engineering is used to convert the raw data into informative inputs for the model. The model is then trained using a portion of the available data and tested using data that were not used during training.

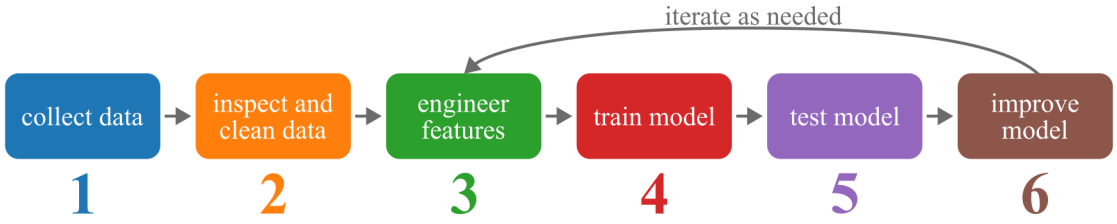


Figure 3.1: A typical machine learning workflow for engineering applications.

The final step is improvement. If the model does not perform well enough, the workflow is repeated by revisiting earlier decisions. This may involve collecting more data, improving the data-cleaning process, changing the engineered features, tuning model parameters, or selecting a different learning algorithm. For this reason, machine learning should be viewed as an iterative process rather than a single pass through the data.

A typical workflow can be summarized as follows:

1. **Collect data:** Gather measurements, simulations, experimental observations, or operational records.
2. **Inspect and clean data:** Check the data for missing values, outliers, incorrect units, noise, and obvious errors.
3. **Engineer features:** Transform the raw data into informative inputs that capture important patterns or physical behavior.
4. **Train model:** Fit a machine learning model using the training data.
5. **Test model:** Evaluate the trained model using data that were not used during training.
6. **Improve model:** Refine the data, features, model settings, or learning algorithm based on the test results.

3.1 Feature Engineering

Feature engineering is the process of transforming raw data into informative inputs that can be used by machine learning algorithms. Carefully designed features can significantly improve model performance and help capture important physical relationships in engineering data.

Figure 3.2 illustrates how raw engineering data can be transformed into a feature vector for machine learning. The features shown in this section are not the only features that can be extracted from data. Other useful inputs may include categorical features, countable features, indicator variables, domain-specific quantities, or features created from simulations and experiments. However, statistical, time-series, and frequency-domain features provide a useful starting point for building informative features from many engineering datasets.

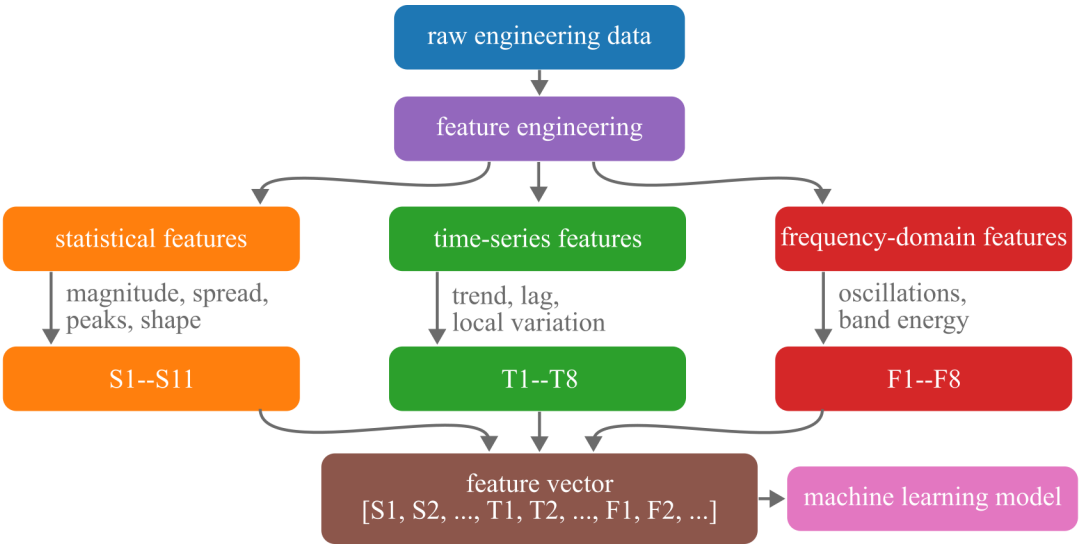


Figure 3.2: Feature engineering transforms raw engineering data into informative model inputs. Features may summarize the statistical properties, time-dependent behavior, or frequency content of the data.

3.1.1 Statistical Features

Statistical features summarize the distribution of values within a dataset, observation, or time window. These features are useful for describing the overall magnitude, variability, symmetry, and peak behavior of the data. Although statistical features are often computed from time-series measurements, they do not necessarily depend on the order of the samples in time.

Let x_i represent the ith value in a set of N measurements, let $\bar{x}$ represent the mean value, and let s represent the sample standard deviation. Table 1 lists a number of statistical features relevant in engineering.

Table 1: Common statistical features for engineering data.

No.	Feature	Equation	Physical Interpretation
S1	Maximum value	$\max(x_i)$	The maximum value captures the largest response in the signal.
S2	Minimum value	$\min(x_i)$	The minimum value captures the smallest response in the signal.
S3	Mean value	$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$	The mean value describes the average level of the signal.
S4	Absolute mean value	$\frac{1}{N}\sum_{i=1}^{N} \lvert x_i \rvert$	The absolute mean value describes the average magnitude of the signal.
S5	Root mean square value	$x_{\text{rms}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}$	The root mean square value describes the effective signal magnitude.
S6	Standard deviation	$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N} (x_i - \bar{x})^2}$	The standard deviation describes how much the signal varies.
S7	Skewness	$\frac{1}{N}\sum_{i=1}^{N} \left(\frac{x_i - \bar{x}}{s}\right)^3$	Skewness describes whether the signal is biased toward one side.
S8	Kurtosis	$\frac{1}{N}\sum_{i=1}^{N} \left(\frac{x_i - \bar{x}}{s}\right)^4$	Kurtosis describes whether the signal contains sharp peaks or outliers.
S9	Shape factor	$\frac{x_{\text{rms}}}{\frac{1}{N}\sum_{i=1}^{N} \lvert x_i \rvert}$	The shape factor compares the effective magnitude to the average magnitude.
S10	Crest factor	$\frac{\max(\lvert x_i \rvert)}{x_{\text{rms}}}$	The crest factor compares the largest peak to the effective signal level.
S11	Impulse factor	$\frac{\max(\lvert x_i \rvert)}{\frac{1}{N}\sum_{i=1}^{N} \lvert x_i \rvert}$	The impulse factor compares the largest peak to the average magnitude.

3.1.2 Time-Series Features

Time-series features describe how a signal changes with time. Unlike statistical features, these features depend on the order of the samples. They are useful when the timing, trend, persistence, or local variation of the signal contains important information.

Table 2 lists a number of time-series features relevant in engineering. Let x_i represent the signal value at time index i, let Δt represent the sampling interval, let k represent a lag, and let w represent the number of samples in a moving window. The local mean over a window is denoted as $\bar{x}_{i,w}$.

Table 2: Common time-series features for engineering data.

No.	Feature	Equation	Physical Interpretation
T1	Lagged value	x_{i-k}	A lagged value captures the recent history of the signal.
T2	First difference	$x_i - x_{i-1}$	The first difference captures the step-to-step change in the signal.
T3	Rate of change	$\frac{x_i - x_{i-1}}{\Delta t}$	The rate of change describes how quickly the signal changes with time.
T4	Moving average	$\frac{1}{w}\sum_{j=i-w+1}^{i} x_j$	The moving average captures the local trend of the signal.
T5	Rolling standard deviation	$\sqrt{\frac{1}{w-1}\sum_{j=i-w+1}^{i}(x_j - \bar{x}_{i,w})^2}$	The rolling standard deviation captures local variability.
T6	Rolling maximum	$\max(x_{i-w+1}, \ldots, x_i)$	The rolling maximum captures recent peak behavior.
T7	Rolling minimum	$\min(x_{i-w+1}, \ldots, x_i)$	The rolling minimum captures recent low-response behavior.
T8	Zero-crossing rate	$\frac{1}{w-1}\sum_{j=i-w+2}^{i} \mathbb{I}(x_j x_{j-1} < 0)$	The zero-crossing rate captures how often the signal changes sign [a].

3.1.3 Frequency-Domain Features

Frequency-domain features describe how the content of a signal is distributed across frequency. These features are commonly extracted by applying a Fourier transform to a measured signal and then summarizing the resulting spectrum. Table 3 lists a number of frequency-domain features relevant in engineering. They are useful when the important behavior in the data is related to oscillations, repeating patterns, or changes in the balance between slow and rapid variations.

Let $P(f)$ represent the power spectrum of the signal at frequency f, and let B_L, B_M, and B_H represent the low-, middle-, and high-frequency bands. The total spectral energy is defined as

$$E_{\text{total}} = \int P(f)\, df. \tag{3.1}$$

In a specific engineering application, the frequency bands should be selected based on the system being studied. When no application-specific bands are known, the usable frequency range can be divided into low-, middle-, and high-frequency regions as a starting point.

[a] $\mathbb{I}(\cdot)$ is an indicator function that equals 1 when the condition is true and 0 otherwise.

Table 3: Common frequency-domain features for engineering time-series data.

No.	Feature	Equation	Physical Interpretation
F1	Dominant frequency	$f_{\text{dom}} = \arg\max_f P(f)$	The dominant frequency captures the strongest periodic behavior.
F2	Spectral centroid	$f_c = \frac{\int f P(f)\, df}{\int P(f)\, df}$	The spectral centroid describes where the spectrum is centered.
F3	Spectral bandwidth	$\sqrt{\frac{\int (f - f_c)^2 P(f)\, df}{\int P(f)\, df}}$	The spectral bandwidth describes how widely the energy is spread.
F4	Low-frequency energy ratio	$\frac{\int_{B_{\text{L}}} P(f)\, df}{E_{\text{total}}}$	The low-frequency energy ratio measures the share of energy in slow behavior.
F5	Middle-frequency energy ratio	$\frac{\int_{B_{\text{M}}} P(f)\, df}{E_{\text{total}}}$	The middle-frequency energy ratio measures the share of energy in mid-range behavior.
F6	High-frequency energy ratio	$\frac{\int_{B_{\text{H}}} P(f)\, df}{E_{\text{total}}}$	The high-frequency energy ratio measures the share of energy in rapid behavior.
F7	High-to-low frequency energy ratio	$\frac{\int_{B_{\text{H}}} P(f)\, df}{\int_{B_{\text{L}}} P(f)\, df}$	The high-to-low frequency energy ratio compares rapid behavior to slow behavior.
F8	Peak-to-total spectral energy ratio	$\frac{P(f_{\text{dom}})}{E_{\text{total}}}$	The peak-to-total spectral energy ratio measures how dominant the largest frequency peak is.

3.2 Training and Testing Data

Training a predictive model on a dataset and then testing it with the same data is a fundamental error in methodology. Such a model could simply memorize the labels of the training samples, achieving perfect performance during training but failing to make any useful predictions on new, unseen data. This phenomenon is known as overfitting. To prevent this, it is standard practice in supervised machine learning to reserve a portion of the available data as a test set, denoted as $\mathbf{X}_{\text{test}}$ and $\mathbf{y}_{\text{test}}$. The function `sklearn.model_selection.train_test_split` in scikit-learn randomly divides arrays or matrices into training and testing subsets.

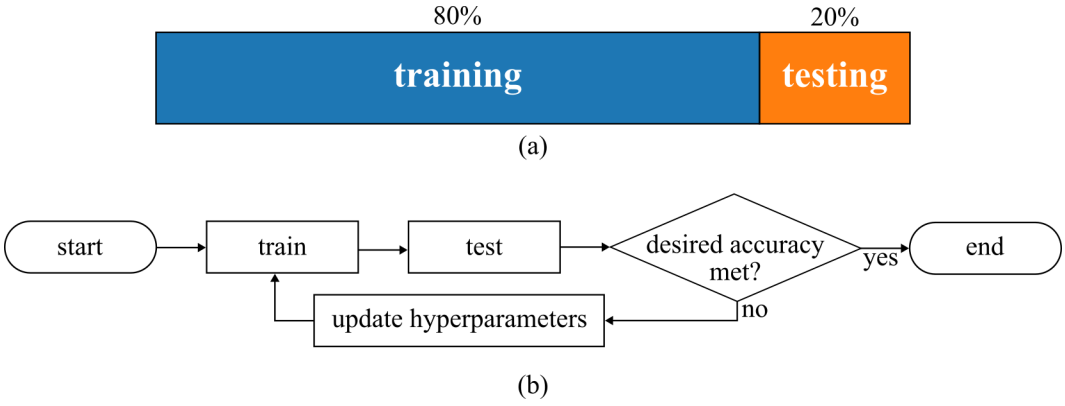


Figure 3.3: Splitting data up into training and testing subsets.

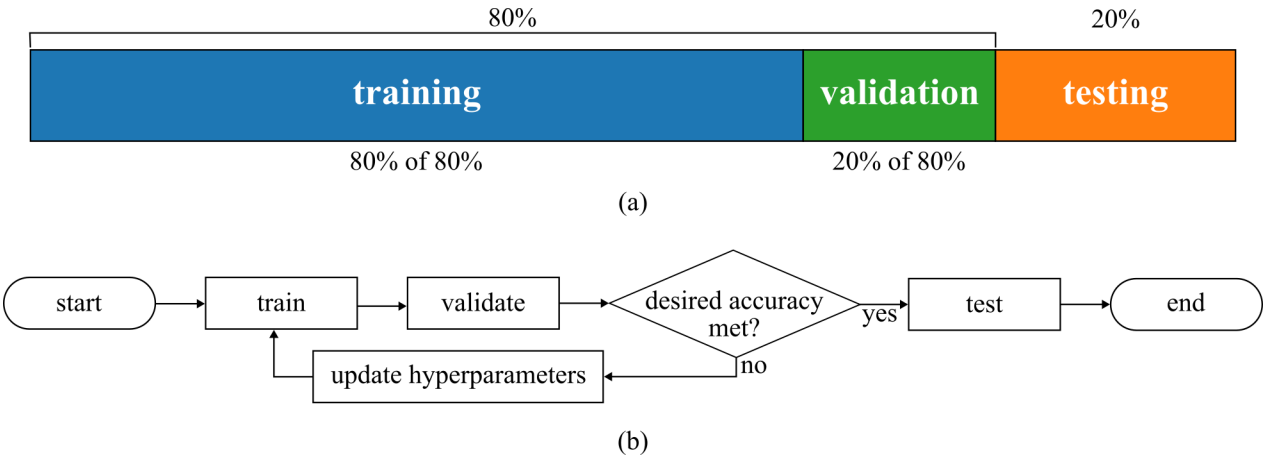


Figure 3.4: Splitting data up into training, validation, and testing subsets.

3.3 Pipelines

Pipelines in scikit-learn streamline the process by sequentially applying a list of transformations followed by a final estimator to a dataset. Employing pipelines allows for the integration of multiple processing steps, which can then be cross-validated together while experimenting with various parameters. Fig. 3.5 illustrates a typical pipeline configuration in scikit-learn.

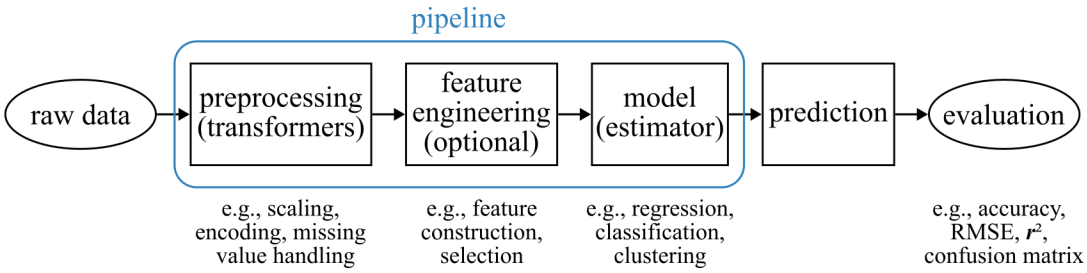


Figure 3.5: Pipeline setup for the automated deployment of pre-processing and modeling steps.

3.4 Learning Curves

Performing Polynomial Regression with a high degree typically allows for a closer fit to the training data compared to simple Linear Regression. For instance, Figure 3.6 demonstrates the application of a 30-degree polynomial model to a set of training data, contrasting it with both a linear model and a quadratic model (2nd-degree polynomial). Observe how the 30-degree polynomial model contorts to closely match the training instances.

- The linear model exhibits underfitting.
- The high-degree Polynomial Regression model drastically overfits the training data.
- Among these, the quadratic model is likely to generalize best.

In Figure 3.6, the appropriateness of the quadratic model is clear since the data was initially generated using such a model. However, in real-world scenarios where the underlying function of the data is unknown, determining the optimal complexity for your model can be challenging. How can you ascertain whether your model is overfitting or underfitting the data?

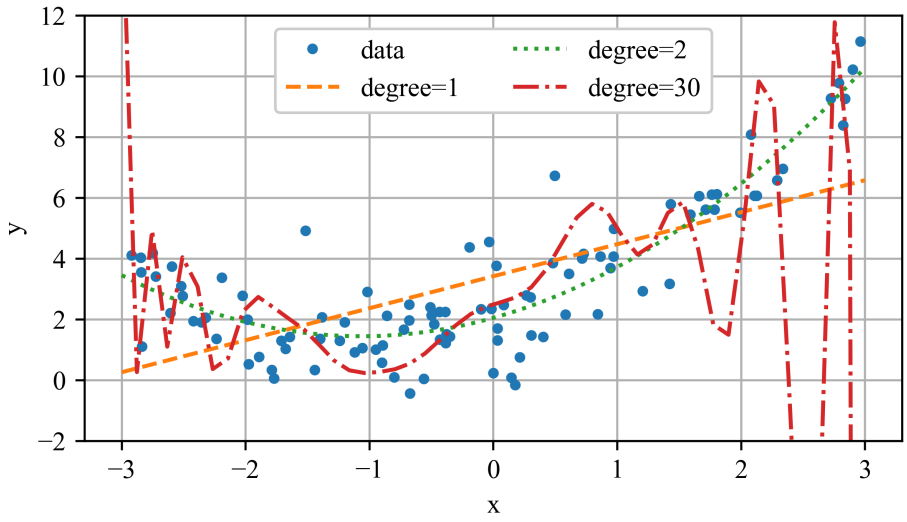


Figure 3.6: Polynomial regression showing underfitting (degree=1), a respectable model fit (degree=2), and overfitting (degree=30).

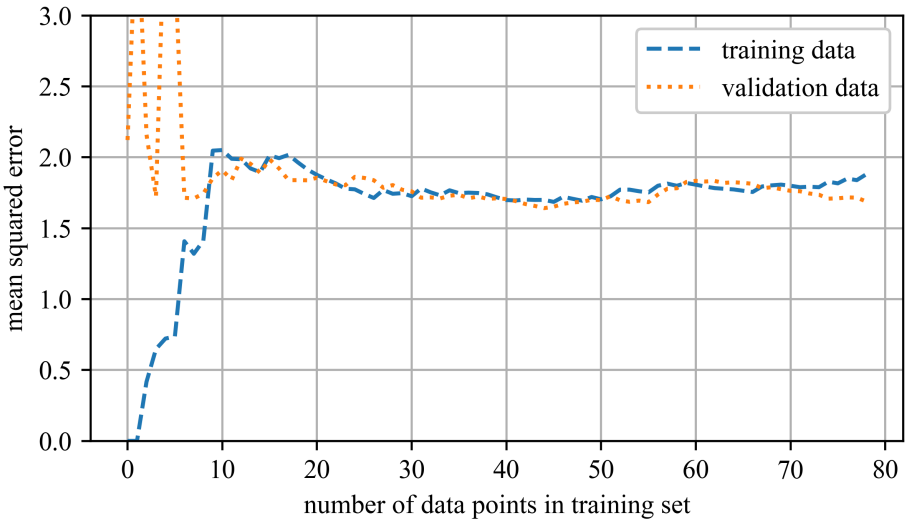


Figure 3.7: Learning curves for the underfitting linear model (degree=1) where underfitting is evident because both curves have plateaued; they are close together and relatively high.

Examining the model's behavior in figures 3.7 and 3.8 on the training set shows a clear trend. With only one or two samples the fit is perfect, so the error curve begins at zero. As additional observations are introduced the model can no longer capture every point exactly, partly because of measurement noise and partly because the underlying relationship is nonlinear, and consequently the training error rises. After a sufficient number of samples the curve flattens out; beyond this plateau adding further data neither markedly lowers nor raises the average error.

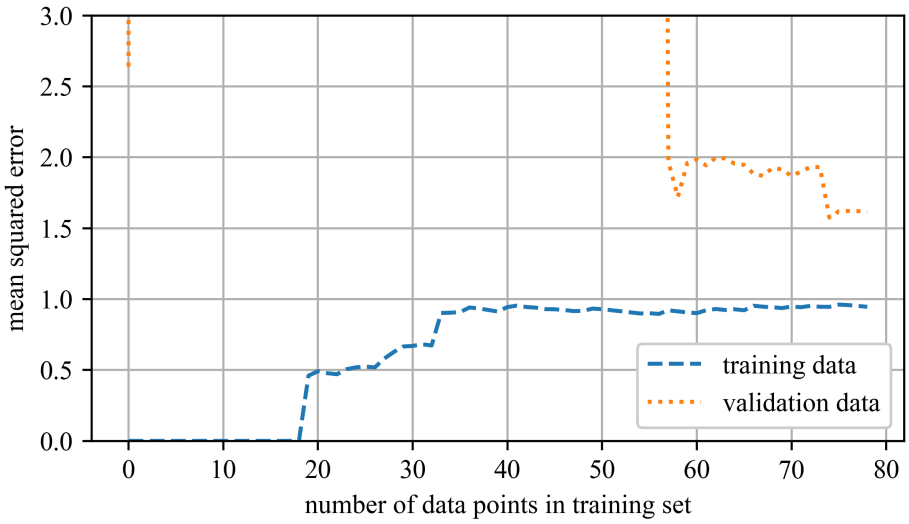


Figure 3.8: Learning curves for the overfitting polynomial model (degree=20) showing a significant gap between the curves, which indicates better performance on the training data than on the validation data.

Inspection of the validation curve in figure 3.8 tells a complementary story. With only a handful of training samples the model cannot generalize, so the validation error starts high. As more examples become available it learns progressively better patterns and the error falls. Eventually, though, the simple linear hypothesis is unable to capture the data's full complexity, causing the decline to flatten out; the validation curve plateaus at nearly the same level as the training curve. Their close proximity and uniformly large values are characteristic of an underfitting model.

Figure 3.8 displays the learning curves for a model fitted with a 20th-degree polynomial. The overall shape resembles the curves seen earlier, yet two key differences stand out. First, the error on the training set is far lower than that achieved by the Linear Regression model, demonstrating the polynomial's extra flexibility. Second, a pronounced gap separates the training and validation curves, which means the model performs much better on the data it has already seen; this divergence is the hallmark of overfitting. Expanding the training set could reduce the gap, but without additional regularization the risk of overfitting would remain.

Next, let's apply the code using a 2nd order polynomial, which accurately captures the essence of the data without underfitting or overfitting. These results are shown in Figure 3.9. Here it can be seen that the training and validation curves again meet after converging, showing a well-fit model.

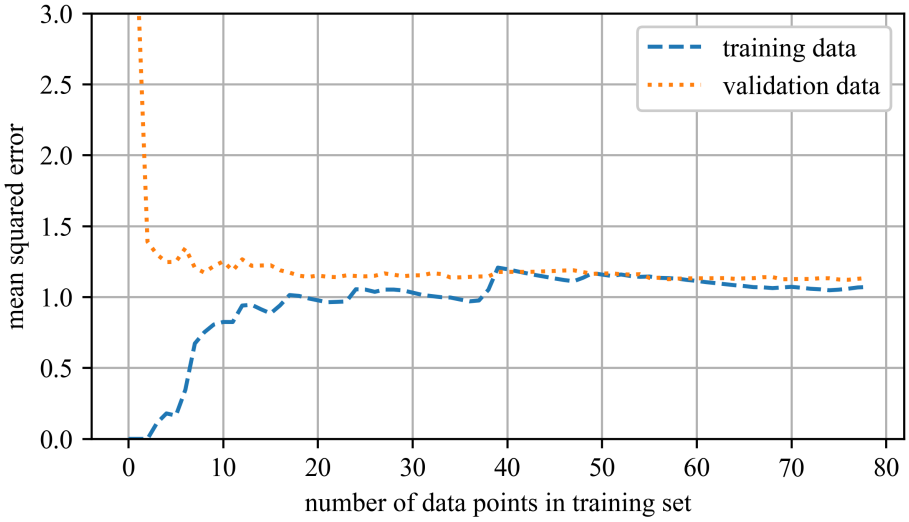


Figure 3.9: Learning curves for the optimal polynomial model (degree=2) showing well-aligned curves that plateau at a relatively low error level.

Example 3.1 Learning Curves
This example compares learning curves for linear and polynomial regression models. By plotting training and validation errors as the training set grows, it illustrates how model complexity impacts bias and variance, and helps identify underfitting and overfitting behavior.

3.5 Regularized Linear Models

Overfitting is frequently curbed by regularizing the model, meaning you introduce constraints that reduce its effective degrees of freedom and make it harder to memories noise. In a polynomial setting you can achieve this simply by choosing a lower-degree polynomial, while in a linear model you typically add a penalty that discourages large coefficient values. This naturally raises the question: how is regularization expressed mathematically?

3.5.1 Ridge Regression

Ridge Regression modifies Linear Regression by adding a regularization term to the cost function, which compels the learning algorithm to fit the data while keeping the model weights as small as possible. The updated cost-function is

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^{n} \theta_i^2. \tag{3.2}$$

The hyperparameter α dictates the degree of regularization. If $\alpha = 0$, Ridge Regression reduces to plain Linear Regression. If α is very high, the model weights are significantly shrunk, resulting in a nearly flat line through the mean of the data. Equation 3.2 outlines the cost function for Ridge Regression. The bias term θ_0 is not regularized, as indicated by the summation starting from $i = 1$.

NOTE

The regularization term is only included during the training phase. After training, the model's performance should be evaluated based on the unregularized performance measure.

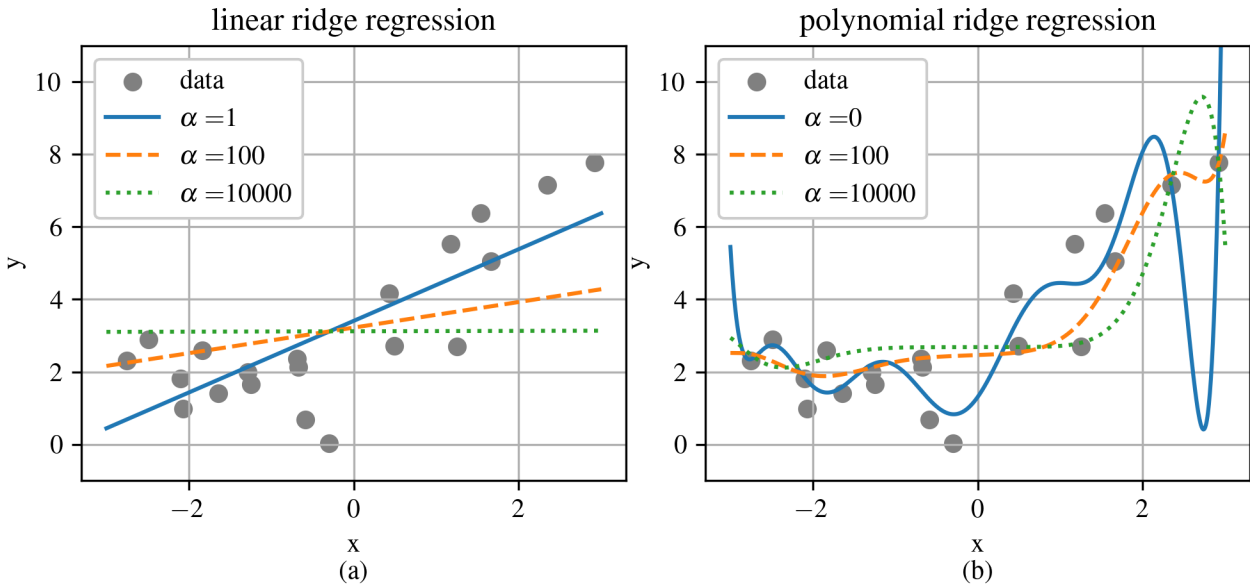


Figure 3.10: Visualization of Ridge Regression models with varying α values.

Figure 3.10 illustrates various Ridge models trained on linear data, showcasing different α value. In Figure 3.10(a), models are purely Ridge Regression, leading to linear predictions. In Figure 3.10(b), the data undergoes Polynomial Regression with Ridge regularization. To do this, the data is first expanded using `PolynomialFeatures(degree=10)`, then scaled with `StandardScaler`, and finally, Ridge models are applied to these transformed features.

Increasing α results in flatter (i.e., more reasonable and less extreme) predictions, thereby reducing the model's variance but increasing its bias. Ridge Regression can be performed using a closed-form solution or through Gradient Descent, similar to Linear Regression. The pros and cons for each method mirror those discussed previously. The closed-form solution, an extension of the normal equation, is given by:

$$\hat{\theta} = (X^\top \cdot X + \alpha A)^{-1} \cdot X^\top \cdot y \tag{3.3}$$

where α influences the regularization and A is an $n \times n$ identity matrix with the top-left value replaced by 0 to exclude the bias term. When using Gradient Descent, the derivative of the cost function guides the adjustment toward optimal model parameters.

Example 3.2 Ridge Regression

This example demonstrates Ridge Regression as a regularized alternative to linear regression. A high-degree polynomial model is fit to noisy data using a pipeline with Ridge regularization, showing how the regularization term reduces overfitting and stabilizes the model.

3.6 Early Stopping

An alternative approach to regularizing iterative learning methods (such as Gradient Descent) is to halt training as soon as the validation error hits its lowest point, a strategy known as early stopping. Figure 3.11 illustrates a complex model, specifically a high-degree Polynomial Regression model, being refined through Batch Gradient Descent. Over the course of training, as epochs progress, the model's prediction error (RMSE) on both the training and validation sets decreases. However, after some time, the validation error ceases to decline and begins to increase, signaling that the model is beginning to overfit the training data. By implementing early stopping, training is terminated the moment the validation error reaches its minimum. Geoffrey Hinton praised this method stating, "Early stopping (is) beautiful free lunch" due to its simplicity [a].

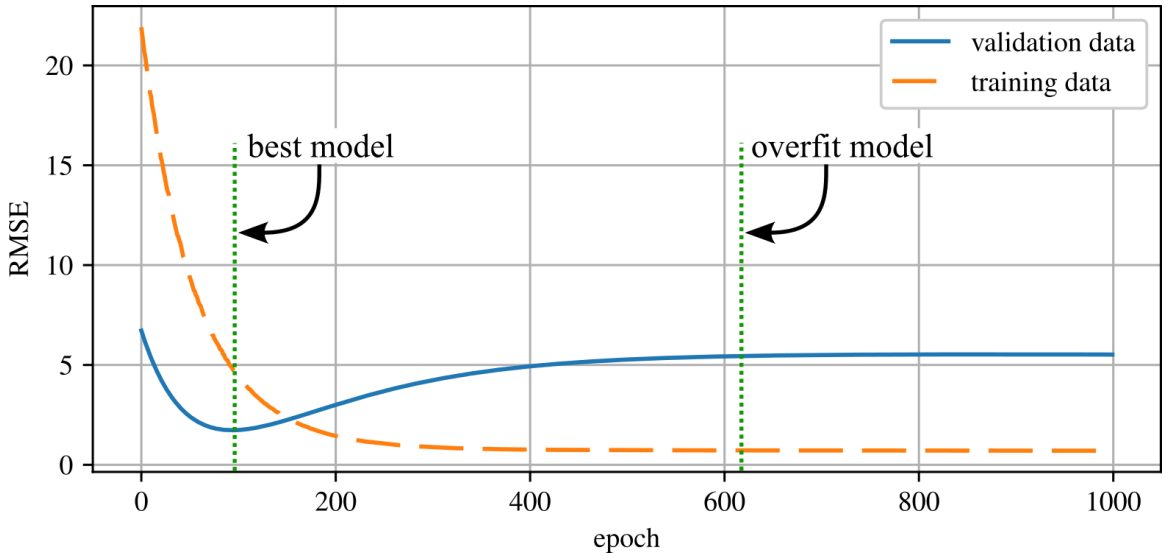


Figure 3.11: An example showing how early stopping can result in a better model as it does not allow for the over-fitting of the training set.

Example 3.3 Early Stopping
This example uses a high-degree polynomial model trained with stochastic gradient descent to demonstrate early stopping. By tracking training and validation errors across epochs, it highlights how halting training early can prevent overfitting and improve generalization.

[a] Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. "Glove: Global vectors for word representation." Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014.

3.7 Examples

Example 3.1

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 3.1
Learning curves
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import pipeline

plt.close('all')

#%% build the data sets
np.random.seed(2) # 2 and 6 are pretty good
m = 100
X = 6 * np.random.rand(m,1) - 3
Y = 0.5 * X**2 + X + 2 + np.random.randn(m,1)
X_model = np.linspace(-3,3)

# plot the data
plt.figure()
plt.grid(True)
plt.scatter(X,Y)
plt.xlabel('x')
plt.ylabel('y')

#%% generate learing curves for a linear model

# build the linear model in SK learn
model = sk.linear_model.LinearRegression()

# split the data into training and validation data sets
# Split arrays or matrices into random train and test subsets
X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)

train_errors, val_errors = [], []
for i in range(1, len(X_train)):
    model.fit(X_train[:i], y_train[:i])
    y_train_predict = model.predict(X_train[:i])
    y_val_predict = model.predict(X_val)

    # compute the error for the trained model
    mse_train = sk.metrics.mean_squared_error(y_train[:i],y_train_predict)
    train_errors.append(mse_train)

    # compute the error for the validation model
    mse_val = sk.metrics.mean_squared_error(y_val,y_val_predict)
    val_errors.append(mse_val)

    # predict model
    y_model = model.predict(np.expand_dims(X_model,axis=1))

    plt.figure('test model')
    plt.scatter(X,Y,s=2, label='data')
    plt.scatter(X_train[:i],y_train[:i], label='data in training set')
    plt.scatter(X_val,y_val, marker='s', label='validation data')
    plt.plot(X_model,y_model,'r--',label='model')
    plt.xlabel('x')
    plt.ylabel('y')
```

```
    plt.legend(loc=2)
    plt.grid(True)
    plt.savefig('test_plots/linear_model_'+str(i))
    plt.close('test model')

plt.figure()
plt.grid(True)
plt.plot(train_errors, "--",label="train")
plt.plot(val_errors, ":", label="val")
plt.xlabel('number of data points in training set')
plt.ylabel('mean squared error')
plt.legend(framealpha=1)
plt.ylim(0,6)
#%% generate learning curves for a polynomial model

model = sk.pipeline.Pipeline((
("poly_features", sk.preprocessing.PolynomialFeatures(degree=20, include_bias=False)),
("lin_reg", sk.linear_model.LinearRegression()),
))

# split the data into training and validation data sets
# Split arrays or matrices into random train and test subsets
X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)

train_errors = []
val_errors = []
for i in range(1, len(X_train)):
    model.fit(X_train[:i], y_train[:i])
    y_train_predict = model.predict(X_train[:i])
    y_val_predict = model.predict(X_val)

    # compute the error for the trained model
    mse_train = sk.metrics.mean_squared_error(y_train[:i],y_train_predict)
    train_errors.append(mse_train)

    # compute the error for the validation model
    mse_val = sk.metrics.mean_squared_error(y_val,y_val_predict)
    val_errors.append(mse_val)

    plt.figure('test model')
    plt.scatter(X,Y,s=2, label='data')
    plt.scatter(X_train[:i],y_train[:i], label='data in training set')
    plt.scatter(X_val,y_val, marker='s', label='validation data')
    y_model = model.predict(np.expand_dims(X_model,axis=1))
    plt.plot(X_model,y_model,'r--',label='model')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend(loc=2)
    plt.grid(True)
    plt.savefig('test_plots/polynominal_model_'+str(i))
    plt.close('test model')

plt.figure()
plt.grid(True)
plt.plot(train_errors, "--",label="train")
plt.plot(val_errors, ":", label="val")
plt.xlabel('number of data points in training set')
plt.ylabel('mean squared error')
plt.legend(framealpha=1)
plt.ylim(0,6)
```

Example 3.2

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 2.4
Ridge Regression
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model
from sklearn import pipeline

plt.close('all')

#%% build the data sets
m = 20
X = 6 * np.random.rand(m, 1) - 3
Y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)

X_model = np.linspace(-3,3,num=1000)
X_model = np.expand_dims(X_model,axis=1)


#%% Perform Ridge Regression

# plot the data
plt.figure()
plt.grid(True)
plt.scatter(X,Y,color='gray')
plt.xlabel('x')
plt.ylabel('y')

# build and plot a linear model
model_linear = sk.linear_model.Ridge(alpha=100, solver="cholesky")
model_linear.fit(X, Y)
y_model_linear = model_linear.predict(X_model)
plt.plot(X_model,y_model_linear,'-',label='linear model')

# build and plot a polynomial model
model_poly = sk.pipeline.make_pipeline(sk.preprocessing.PolynomialFeatures(10),
                                       sk.linear_model.Ridge(alpha=100, solver="cholesky"))
model_poly.fit(X, Y)
y_model_poly = model_poly.predict(X_model)
plt.plot(X_model,y_model_poly,'-',label='polynomial model')

plt.legend()
```

Example 3.3

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 3.3
Early Stopping
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk

plt.close('all')

#%% build the data sets

# use 6 to help give a smooth curve that makes the case for early stopping
np.random.seed(6)

m = 20
X = 6 * np.random.rand(m, 1) - 3
Y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)

# plot the data
plt.figure()
plt.grid(True)
plt.scatter(X,Y,color='gray')
plt.xlabel('x')
plt.ylabel('y')

X_model = np.linspace(-3,3,num=1000)
X_model = np.expand_dims(X_model,axis=1)

X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)

#%% perform early stopping


# prepare the data
poly_scaler = sk.pipeline.Pipeline([("poly_features", sk.preprocessing.PolynomialFeatures
(
    degree=90, include_bias=False)), ("std_scaler", sk.preprocessing.StandardScaler())])
X_train_poly_scaled = poly_scaler.fit_transform(X_train)
X_val_poly_scaled = poly_scaler.fit_transform(X_val)

# set up the model, not that by setting max_iter=1 it will only train one epoch
model = sk.linear_model.SGDRegressor(max_iter=1, tol=0,learning_rate="constant"
    ,eta0=0.0005,penalty=None,warm_start=True)


# Train the model in a loop to build the data set to investigate the benefit of early
stopping
val_errors = []
train_errors = []
for epoch in range(1000):
    model.fit(X_train_poly_scaled, y_train.ravel()) # continues where it left off
    y_val_predict = model.predict(X_val_poly_scaled) # Predict the target values
    y_train_predict = model.predict(X_train_poly_scaled) # Predict the target values
    val_error = sk.metrics.mean_squared_error(y_val, y_val_predict)  # Calculate error
    train_error = sk.metrics.mean_squared_error(y_train.ravel(), y_train_predict)  #
    Calculate error
    val_errors.append(val_error)
    train_errors.append(train_error)
```

```
65
66  # plot the early learning curves, you may have to plot this a few times to get
67  # a set of curves that shows strong results
68  plt.figure()
69  plt.grid(True)
70  plt.plot(val_errors,label='validation data')
71  plt.plot(train_errors,'--',label='training data')
72  plt.ylabel('RMSE')
73  plt.xlabel('epoch')
74  plt.legend()
75
76
77
78
79
80
```